pycognition: Eine Dual-Prozess-Kognitionsarchitektur zur Integration deterministischer prozeduraler Ausführung und deliberativer LLM-Synthese

Oleh Riabtsev
Referat Forschung, Nachwuchsförderung und Transfer (RFNT), Universität Vechta, Deutschland
ORCID: 0009-0003-7848-6418 | Softwarelizenz: LGPL-3.0-or-later

KURZZUSAMMENFASSUNG

Die vorliegende Arbeit formalisiert das Framework **pycognition**, welches die psychologische Dual-Prozess-Theorie für autonome Softwareagenten adaptiert. Bestehende Agentensysteme leiten deterministische und iterative Aufgaben monoton über stochastische Schnittstellen großer Sprachmodelle (LLMs), was zu Latenzaufblähungen und hohen Tokenkosten führt. pycognition trennt den Ablauf in ein reflexives System 1 (Python-Skripte) und ein deliberatives System 2 (Frontier-Modelle). Über die automatisierte Kompetenzkristallisation (Skill Crystallization) werden Lösungswege in typisierten Code überführt, via AST geprüft und in einer `pytest`-Sandbox verifiziert. Eine Evaluierung über 100 Aufgaben belegt die Schutzwirkung der Sandbox (36,2% Abweisung fehlerhafter Codes) und etabliert ein realistisches dreistufiges Leistungsmodell. Unter expliziter Berücksichtigung des Skripterstellungsaufwands (im Mittel 7.192,20 ms) amortisiert sich die Investition für eine Fertigkeit bereits beim 4. Aufruf (ideal) bzw. 5. Aufruf (unter Berücksichtigung von Sandbox-Rückweisungen). Danach wird eine marginale Beschleunigung von 11.326,7x erzielt. Auf Systemebene sank die Antwortzeit für algorithmische Aufgaben um 44,7% (155,46 s vs. 281,13 s), während der Tokenverbrauch sofort um 38,0% reduziert wurde.

Schlüsselwörter: Kognitive Architekturen, Autonome Agenten, Dual-Prozess-Theorie, Programmsynthese, AST-Validierung, Amortisierte Inferenz, JIT-Kompilierung, Break-Even-Point, pycognition.

1. EINLEITUNG

Der produktive Einsatz autonomer Softwareagenten leidet unter der Ineffizienz stochastischer Ablaufumgebungen für deterministische Aufgaben. Standardmuster (ReAct [6], Plan-and-Solve [9], AutoGen [10]) leiten Datenparsing, Formatierungen und Berechnungen wiederholt über Cloud-APIs. Dies verursacht skalierende Tokenkosten, Latenzen von bis zu 5.000 ms und stochastische Regressionsrisiken.

Die Dual-Prozess-Theorie [1], [2] bietet hierfür die funktionale Blaupause: **System 1** verarbeitet automatisierte Routinen extrem schnell, während **System 2** ressourcenintensive Analysen übernimmt. **pycognition** realisiert diesen Übergang als lernender JIT-Compiler: Bei neuen Aufgabenklassen investiert System 2 Rechenzeit in die Synthese deterministischer Python-Module, verifiziert diese und überführt sie in das Gedächtnis.

2. VERWANDTE ARBEITEN

Die Trennung von deklarativem und prozeduralem Gedächtnis prägte bereits ACT-R [3] und Soar [4]. Das Paradigma der amortisierten Inferenz [15] begründete den Ersatz iterativer Suchprozesse durch gecachte Funktionsauswertungen. Moderne Codegenerierungsmodelle [12], [13] wie in Voyager [11] zeigten die Möglichkeit der Kompetenzakkumulation, ließen jedoch standardisierte AST-Sicherheitsprüfungen und automatisierte Testsynthesen [14] vermissen.

3. METHODIK UND SYSTEMARCHITEKTUR

Eine Rechenaufgabe $T = \langle \mathrm{id}, p, x, \mu \rangle$ wird über den `MetaArbiter` anhand der Ähnlichkeitsmetrik $\gamma(T, \Sigma)$ bei $\theta = 0.55$ arbitriert. Erfolgreiche Deliberationen überführt der Operator Ψ in Module, gesichert durch statische AST-Filterung und dynamische Tests in einer Subprozess-Sandbox ($t = 15$ s).

4. IMPLEMENTIERUNG UND EMPIRISCHE EVALUIERUNG

4.1. Dreistufiges Latenz- und Amortisationsmodell (N = 100)

- **Stufe 1. Marginale Ausführungslatenz:** Ein kompiliertes Skript in System 1 benötigt im Mittel $\overline{L}_{S_1} = 0.292$ ms. Ein Basis-LLM-Aufruf ohne Synthese erfordert $L_{\text{base}} = 3\,307.41$ ms. Dies ergibt eine marginale Beschleunigung von **11.326,7x** (19.103,9x über alle System-2-Aufrufe).
- **Stufe 2. Skripterstellungsaufwand und Break-Even-Point:** Die Synthese und Sandbox-Verifikation erfordert im Mittel $L_{\text{cryst}} = 7\,192.20$ ms (Gesamtlatenz des Kaltstarts: 10 499.60 ms).
  Die Amortisationsformel für k Aufrufe lautet:

$$k \geq 1 + \frac{L_{\text{cryst}} - \overline{L}_{S_1}}{L_{\text{base}} - \overline{L}_{S_1}} = 1 + \frac{7\,192.20 - 0.292}{3\,307.41 - 0.292} \approx 3.175 \Longrightarrow k = 4 \text{ Aufrufe}$$

  Unter Einbeziehung der 36,2% Sandbox-Rückweisungen ($L_{\text{cryst,eff}} = 11\,267.78$ ms) liegt der effektive Break-Even-Point bei $k_{\text{eff}} \geq 4.407 \Longrightarrow k = 5$ Aufrufen.
  Entwicklung der effektiven Beschleunigung: $S(1) = 0.31\times$; $S(2) = 0.63\times$; $S(4) = 1.26\times$ (Gewinnschwelle); $S(10) = 3.15\times$; $S(50) = 15.73\times$; $S(100) = 31.41\times$.
- **Stufe 3. Systemebene über 100 Aufgaben:**
  Reine LLM-Ausführung: 471,66 s (281,13 s algorithmisch). In pycognition sank die Antwortzeit für algorithmische Aufgaben im asynchronen Betrieb auf **155,46 s (1,81-fache Beschleunigung, -44,7% Zeitaufwand)**. Synchron beanspruchten die 47 Syntheseläufe 338,03 s (Gesamtzeit 493,49 s auf Algorithmen). Im 4. Quartil erreichte die synchrone Ausführungsrate **0,96x der Baseline (2,84x reine Antwortbeschleunigung)**. Der systemweite Break-Even-Horizont konvergiert bei $N^* \approx 207$ Aufgaben.

Tabelle 2. Realistische Leistungskennzahlen (N = 100 Aufgaben)

Leistungsdimension	Reiner LLM-Betrieb (Baseline)	pycognition Dual-Process	Empirischer Effekt
Marginale Latenz System 1	3.307,41 ms (algo)	0,292 ms (System 1)	Sub-millisekundengenau
Syntheselatenz (Skripterstellung)	0 ms	7.192,20 ms	JIT-Investitionskosten
Kaltstart-Latenz (Gesamt)	3.307,41 ms	10.499,60 ms	Temporärer Mehraufwand (0,31x)
Break-Even-Point eines Skills	Nicht anwendbar	k = 4 (ideal) / k = 5 (effektiv)	Amortisation nach 4–5 Aufrufen
Marginale Beschleunigung	1,0x	11.326,7x (algo) / 19.103,9x (ges.)	Asymptotische Grenze
Antwortzeit 85 algorithmische Aufgaben	281,13 s	155,46 s (asynchron)	1,81x Beschleunigung (-44,7%)
Synchrone Gesamtlaufzeit	281,13 s	493,49 s	Investitionsphase bei N=100
Systemweiter Break-Even-Horizont N*	Nicht anwendbar	N* ≈ 207 Aufgaben	Vollständige Cluster-Amortisation
Tokenverbrauch (100 Aufgaben)	87.061 Tokens	53.978 Tokens	-38,0% sofortige Ersparnis
System 1 Anteil in Quartil 4	0,0%	71,4% (15 von 21 algorithmisch)	Starke Konsolidierung
Sandbox-Validierungsrate	Nicht anwendbar	63,8% (30 genehmigt / 17 geblockt)	100% fehlerfreies System 1

5. DISKUSSION UND LIMITATIONEN

Die Abweisung von 36,2% der synthetisierten Module belegt die Schutzwirkung der Sandbox gegen Inkonsistenzen bei Freiheitsgraden (6 Z-Score-Fälle), fehlende Oktettprüfungen (4 IPv4-Fälle) und Rundungsfehler. Die explizite Berücksichtigung des Syntheseaufwands (7,19 s) liefert ein realistisches Fundament für produktive Agenten, bei denen die Codesynthese asynchron im Hintergrund erfolgen sollte. Die TF-Vektorsuche zeigte zudem semantische Alias-Effekte und sprachübergreifende Vektorverdünnungen, was den künftigen Einsatz dichter Embeddings motiviert.

6. FAZIT UND AUSBLICK

pycognition belegt die Praxistauglichkeit des dualen Kognitionsansatzes unter realen Synthesebedingungen: Die anfängliche Entwicklungszeit amortisiert sich nach 4–5 Aufgabenwiederholungen, senkt die Antwortzeit um 44,7% und spart sofort 38,0% der Tokenkosten ein. Künftige Arbeiten fokussieren auf selbstheilende Syntheseschleifen via Pytest-Tracebacks und dichte Embeddings.

7. LITERATURVERZEICHNIS

[1] D. Kahneman, *Thinking, Fast and Slow*. New York, NY, USA: Farrar, Straus and Giroux, 2011.

[2] J. S. B. T. Evans and K. E. Stanovich, "Dual-process theories of higher cognition: Advancing the debate," *Perspectives on Psychological Science*, 2013.

[3] J. R. Anderson, *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, 2007.

[6] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[8] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[11] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," in *Proc. NeurIPS*, 2023.

[15] S. J. Gershman and N. D. Goodman, "Amortized inference in probabilistic reasoning," in *Proc. CogSci*, 2014.